

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

MASTER THESIS FALL 2024

MASTER IN APPLIED MATHEMATICS

**Exploring Deep Learning Models:
Experimental Insights into Linear Recurrent Units**

Author:
Seorim PARK

Supervisor:
Lénaïc CHIZAT
Pierre MARION

EPFL

Contents

1	Introduction	1
2	Theoretical background of the deep learning models	1
2.1	Recurrent Neural Network	1
2.2	Linear recurrent unit	2
2.3	State Space models	5
2.4	Transformers	5
2.5	A meta-model	6
3	Experimental results	7
3.1	Main framework of the model	7
3.2	Results of the experiments	8
3.2.1	Depth	10
3.2.2	Hidden dimension of the MLP	11
3.2.3	Learning rate	12
3.2.4	Ablation studies	14
3.2.5	Change in the structure	15
3.2.6	Layer-wise parameterization	16
3.3	Summary of the results	17
4	Discussion	20
A	Proofs	23
A.1	Proof of discretization in Section 2.3	23
B	Additional experimental results	24
B.1	Results of the layer-wise parameterization	24

1 Introduction

Deep neural networks are models that involve multiple layers of complex neural network structures to work on complex tasks, such as classification or generation of datasets such as texts or images [LeCun et al., 2015]. Recent research on deep neural network models has gathered significant attention as the model serves as a cornerstone in the application of artificial intelligence on tasks across diverse fields. For instance, AlphaFold [DeepMind, 2018], developed by DeepMind in 2018, is an artificial intelligence program capable of predicting the protein’s three-dimensional structure from its amino acid sequence. Its prediction model is built upon pairformers, a derivation of transformers, which is a deep learning technique [Abramson et al., 2024]. Another notable example is ChatGPT [OpenAI, 2022], an artificial intelligence chatbot based on transformers developed by OpenAI in 2022, which garnered all the interest due to its exceeding performance on complex tasks [Bansal et al., 2024] [Oh et al., 2023].

The potential of deep neural networks lies in two core principles [Bengio et al., 2013] [Zhang et al., 2017]:

1. The ability to concisely compress or transform the input data to capture the essential information and patterns by focusing on the dependencies existing within the data.
2. The integration of multiple neural network layers to be capable of approximating complex representations.

While deep learning methods demonstrate impressive performance across diverse tasks, incorporating complex structures into the learning model leads to a sharp increase in computation costs. These settings necessitate machines with high computational power and excessive energy consumption [Justus et al., 2018]. Depending on the structure, models can involve billions of parameters, and high-dimensional inputs to the scale of billions can quickly reach the memory and computational limit for any existing machines. This problem has become apparent with the emergence of transformers [Vaswani et al., 2017], which, though widely used across different domains, are known to be computation-wise expensive [Puvis de Chavannes et al., 2021].

A possible path to tackle this problem is to develop a model that can balance performance and efficiency, in other words, to achieve the most accurate predictions with minimal computations. Several approaches have been suggested to replace models with high costs. Some of the proposed models begin with the basic structure of Recurrent Neural Networks (RNNs) and then refine and modify a part of the structures to enhance efficiency. Others aim at reducing the cost of transformers by suggesting more efficient methods of input data compression. In particular, linear recurrent units (LRUs) have shown unexpectedly strong performance. Building on these advancements, we present empirical investigations on linear recurrent units by focusing on the influence of the architecture and the hyperparameters on model performance.

2 Theoretical background of the deep learning models

In this section, we present different existing deep neural network models for sequential input and discuss how those models deal with the trade-off problem of balancing between reducing the computation cost and maintaining the performance. We define the input data $\mathbf{x} \in \mathbb{R}^{N \times L}$, composed of N tokens of length L each, and the output $\mathbf{y} \in \mathbb{R}^S$. The goal of the deep learning models is to obtain an approximation $\hat{f}$ of the function f such that $f(\mathbf{x}) = \mathbf{y}$ for $\mathbf{x} \in \mathcal{X}$ and $\mathbf{y} \in \mathcal{Y}$ with $\mathcal{X}$ and $\mathcal{Y}$ the domain of the input data and the output data respectively.

2.1 Recurrent Neural Network

Recurrent Neural Network [Rumelhart et al., 1986] (referred to as RNN) is a method to transform the input data by incorporating hidden states, which play the role of memorizing and mixing the previous input tokens to produce the output. Let x_k be the k^{th} token of the input $\mathbf{x}$, with $k \in [1, N]$ an integer. Then RNN computes the hidden state $h_k \in \mathbb{R}^D$ by combining the information from the hidden state of the previous token h_{k-1} with the information of the current k^{th} token x_k . The output y_k is then

computed by mixing the information from the computed hidden state h_k and the current token x_k . The process can be summarized as follows:

$$h_k = \sigma(Ah_{k-1} + Bx_k), \quad (1a)$$

$$y_k = Ch_k + Dx_k \quad (1b)$$

where $A \in \mathbb{R}^{D \times D}$, $B \in \mathbb{R}^{D \times L}$, $C \in \mathbb{R}^{S \times D}$ and $D \in \mathbb{R}^{S \times L}$ are weight matrices, and $\sigma : \mathbb{R}^D \rightarrow \mathbb{R}^D$ a non linear function. The output $y = \{y_1, \dots, y_N\}$ of the RNN is of size $N \times S$, and its dimension can be adjusted by a linear transformation. In this RNN model, the weight matrix A of equations (1) plays an important role as it determines how much information to store to h_k from the previous tokens $x_{[1, k-1]}$. σ denotes the non-linearity in the model, replaced by an activation function such as ReLU or tanh.

Although RNNs have suggested a reasonable method of mixing the information in the input data, several drawbacks exist. Before all, the model's performance is hindered by the vanishing gradient problem [Hochreiter, 1998]. Depending on the eigenvalues of the matrix A from equations (1), the previous tokens $(x_i)_{i=1}^{k-1}$ can be under-represented or over-represented when computing y_k .

Indeed, computing the gradient of h_k with respect to h_n with $n \in \mathbb{N}$ between 1 and k [Pascanu et al., 2013], we get:

$$\frac{\partial h_k}{\partial h_n} = \frac{\partial h_k}{\partial h_{k-1}} \frac{\partial h_{k-1}}{\partial h_{k-1}} \cdots \frac{\partial h_{n+1}}{\partial h_n}$$

As $\frac{\partial h_i}{\partial h_{i-1}} = \sigma'(Ah_{i-1} + Bx_i)A$, we get:

$$\frac{\partial h_k}{\partial h_n} = A^{k-n-1} \prod_{i=n+1}^k \sigma'(Ah_{i-1} + Bx_i)$$

If $k-n$ increases, the gradient $\frac{\partial h_k}{\partial h_n}$ disappears if the eigenvalues of A are smaller than one, and explodes when the eigenvalues of A are greater than one. In other words, the hidden state at step n gives a minor or exaggerated impact when computing the hidden state at step k as n gets smaller. Therefore, RNN is not appropriate for modeling long-range dependencies as it only captures the information from the neighboring tokens or the initial tokens of the input data. For instance, in an image classification task, when the eigenvalues of the matrix A are smaller than the unit, the hidden states only keep the information from dozens of previous pixels for each token (pixel). Thus, the model cannot compute the output by incorporating the information from the whole image using RNNs. Another significant issue raised by the structure of RNN is the inefficiency created by the non-linear function. The non-linear function σ impedes the possibility to compute the output token y_k in parallel. The sequential computation of the hidden state and the output of step N fixes the computation cost of RNN to $O(N)$.

2.2 Linear recurrent unit

The linear recurrent unit (referred to as LRU) [Orvieto et al., 2023] is a model based on RNN. Although maintaining the same architecture as RNN, the difference comes in the absence of non-linear function σ from equation (1), which can be summarized as follows:

$$h_k = Ah_{k-1} + Bx_k, \quad (2a)$$

$$y_k = Ch_k + Dx_k \quad (2b)$$

with $A \in \mathbb{R}^{D \times D}$, $B \in \mathbb{R}^{D \times L}$, $C \in \mathbb{R}^{S \times D}$ and $D \in \mathbb{R}^{S \times L}$. Contrary to the belief that layers with non-linear activation functions have better performance as they are Turing-complete [Kilian and Siegelmann, 1996], the paper [Orvieto et al., 2023] showed that linear RNN can be more performant than the latter when increasing the depth of the linear RNN in pair with one hidden-layered Multi-Layer Perceptron (MLP). In other words, the Turing incompleteness of the linear RNN can be solved by sequentially adding multiple layers of the linear RNN and MLP pairs to the model architecture.

The authors have further mathematically proved the possibility of fully recovering the original input data from the hidden state using linear RNN [Orvieto et al., 2024] combined with an MLP. They demonstrated the possibility to perform a lossless compression of the input information and an

accurate fit to complex functions, provided that the dimension of the hidden state and the number of neurons in the hidden layer of the MLP is sufficient. Let $\mathbf{x} = (x_i)_{i=1}^N$ and $\mathbf{y} = (y_i)_{i=1}^N$. The model tries to approximate a non-linear continuous function $f = (f_k)_{k=1}^N$, which exhibit a causal structure such that $f_k((x_i)_{i=1}^k) = (y_i)_{i=1}^k$ where each y_k only depend on the previous inputs $(x_i)_{i=1}^k$. The function f is approximated by $\hat{f} = g \circ R \circ e$, where $e : \mathbb{R}^L \rightarrow \mathbb{R}^E$ is the linear embedding function acting for each token x_k , $R : \mathbb{R}^E \rightarrow \mathbb{R}^D$ the linear RNN and $g : \mathbb{R}^D \rightarrow \mathbb{R}^S$ the MLP. The linear RNN is defined as follows:

$$h_k = Ah_{k-1} + Bx_k$$

Provided that the weight matrix A is diagonalizable, let $A = PAP^{-1}$, then we get:

$$\begin{aligned} h_k &= PAP^{-1}h_{k-1} + Bx_k \\ (P^{-1}h_k) &= \Lambda(P^{-1}h_{k-1}) + (P^{-1}B)x_k \end{aligned}$$

if we let $h_k \leftarrow P^{-1}h_k$ and $B \leftarrow P^{-1}B$, we get:

$$h_k = \Lambda h_{k-1} + Bx_k \quad (3)$$

Unfolding the recurrence in equation (3), we can rewrite h_k as:

$$h_k = \sum_{i=1}^k \Lambda^{i-1} Bx_{k-i+1} \quad (4)$$

The authors showed that h_k can be written as the product of a Vandermonde matrix and the input data $(x_i)_{i=1}^k$ when letting $B = (1, 1, \dots, 1)^T$:

$$h_k = \begin{pmatrix} \lambda_1^{k-1} & \dots & \lambda_1 & 1 \\ \dots & \dots & \dots & \dots \\ \lambda_D^{k-1} & \dots & \lambda_D & 1 \end{pmatrix} \mathbf{x}_{1:k} \quad (5)$$

Let V_k the Vandermonde matrix of equation (5), then using the pseudo inverse V_k^+ of V_k , we can write:

$$\mathbf{x}_{1:k}^T = V_k^+ h_k^T \quad (6)$$

The equation (6) shows that it is possible to do a lossless compression of the tokens from 1 to k of the input data when computing the hidden state h_k via linear RNN if the Vandermonde matrix is invertible, which means that it must have all distinct $(\lambda_i)_{i=1}^D$. This condition holds if $D \geq \dim(\mathbf{x})$. As a result, the upcoming MLP layer function g has full access to the information from $\mathbf{x}$ via the linear RNN to approximate f if the condition is satisfied.

In addition, one hidden-layered MLPs are known to be universal approximators of non-linear functions [Barron, 1993]. Let us recall the definition of a Barron function:

Definition 1 (Barron function). *Let $\mathcal{F}(\omega)$ be the Fourier transform of $f : \mathbb{R}^n \rightarrow \mathbb{R}$. f is a Barron function if $C_f = \int_{\mathbb{R}^n} \|\omega\|^2 |F(\omega)| d\omega < \infty$. We denote C_f as the Barron constant of the function f .*

This definition induces the following theorem [Barron, 1993]:

Theorem 2.1. *Let g be a one hidden-layered MLP with H hidden neurons and with a sigmoidal activation function on the hidden layer. Let f a continuous Barron function and C_f its Barron constant, then there exist some g such that $\sup_{\|x\| < r} |f(x) - g(x)| < \epsilon$ if $H \geq 2r^2 C_f^2 \epsilon^{-2}$.*

Therefore, unlike previous studies showing the limited expressivity of linear RNN on non-linear functions, the authors showed that with a sufficient number of hidden state dimensions D , the linear RNN can perfectly memorize the input data, and the non-linearity can be captured when coupled with a one-hidden-layered MLP with the necessary number of hidden neurons.

Experimental results justify its performance. Keeping the multiple sequential layers of the LRU and MLP pair, the authors of [Orvieto et al., 2023] focus on three main elements to enhance the accuracy: the initialization and the use of complex spaces in LRU, the stabilization of the weight matrix Λ via exponential parameterization and the normalization. First, the model uses the complex space to sample the values of the weight matrices A , B , C , and D from equation (2). As shown in

equation (3), it is possible to replace the matrix A with a diagonal matrix Λ containing the eigenvalues of A . The model uses the Glorot initialization to sample the initial value of Λ : the initial values are sampled from a normal distribution of mean 0 and variance $1/D$, where D is the number of eigenvalues of Λ .

The stability of Glorot initialization is explained by the strong circular law [Ginibre, 1965]:

Theorem 2.2 (Strong circular law). *Let μ_D be the empirical spectral measure of A_D , where A_D is a real $D \times D$ matrix with i.i.d. Gaussian entries, each with zero mean and variance $1/D$. Then μ_D converges almost surely as D tends to infinity to the uniform probability measure on $\{|z| \leq 1\} \subseteq \mathbb{C}$.*

The theorem shows that the Glorot initialization is equivalent to sampling diagonal values of Λ from a unit disk on $\mathbb{C}$, which ensures an approximately stable initialization. Moreover, we can change the sampling system using the definition of exponential of a complex number: $\exp(-\nu + i\theta) = \exp(-\nu)(\cos(\theta) + i\sin(\theta))$. To simplify the scheme and narrow down the spectrum to a ring between the circles of radii r_{min} and r_{max} , the paper uses the following lemma:

Lemma 2.3. *Let u_1, u_2 be independent uniform random variables on the interval $[0, 1]$. Let $0 \leq r_{min} \leq r_{max} \leq 1$. Compute $\nu_k = -\frac{1}{2} \log(u_{k1}(r_{max}^2 - r_{min}^2) + r_{min}^2)$ and $\theta_k = 2\pi u_{k2}$. Then $\exp(-\nu_k + i\theta_k)$ is uniformly distributed on the ring in $\mathbb{C}$ between circles of radii r_{min} and r_{max} .*

The paper shows that tuning the right r_{min} and r_{max} improved the performance of LRU on classification tasks such as CIFAR-10, ListOps, or Pathfinder.

Equations (2) show that the eigenvalues of Λ must have a magnitude closer to the unit to solve the vanishing gradient problem. However, it is observed that these eigenvalues make the training unstable. To solve the stability problem, the paper used another positive non-linearity. ν is initialized differently, by taking $\nu^{\log}$ as the parameter to optimize initialized by $\nu^{\log} = \log \nu$ and computing $\lambda_j = \exp(-\exp(\nu^{\log}) + i\theta_j)$.

In addition, tuning r_{min} and r_{max} closer to the unity resulted in the blow-up of the training loss. Indeed, the paper shows that the expectation of $|h_k|^2$ tends to $\frac{1}{1-|\lambda|^2}$ when k tends to infinity, with $\lambda = \{\lambda_1, \lambda_2, \dots, \lambda_k\}$. As the value blows up when λ approaches to 1, the model adds a normalization parameter $\gamma^{\log} \in \mathbb{R}^N$ defined as $\gamma^{\log} = (\log(\sqrt{1-|\lambda_i|}))_{i=1}^N$ initialized element-wise.

The final LRU model is as follows:

$$\begin{aligned} h_k &= \Lambda h_{k-1} + \exp(\gamma^{\log}) \odot (Bx_k), \\ y_k &= Ch_k + Dx_k \end{aligned}$$

where Λ is the diagonal matrix of eigenvalues written as $\Lambda = \text{diag}(\exp(-\nu + i\theta))$ with ν and θ learnable parameters.

The linearity of the model supports its efficiency. Indeed, the use of parallel scan [Blelloch, 1990] on recurrent models can significantly reduce the computation cost to $O(N/p + \log p)$ with p the number of cores, just by removing the non-linear function σ in equation (1). The parallel scan (or the prefix sum) uses parallel computing to find the sum of a sequence of numbers. Figure (2.1) shows an example of the application of the parallel scan.

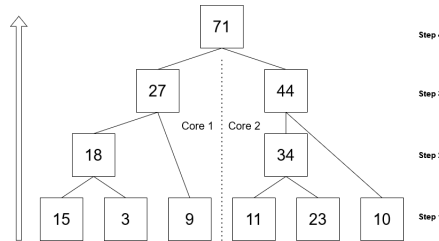


Figure 2.1: An example of the application of prefix sum on a sequence of 6 numbers with two cores.

As the recurrence from equation (3) can be written explicitly as equation (4), the values of h_k can be computed in parallel by using the parallel scan algorithm on the sums. This process breaks down the computation cost from $O(N)$ to $O(N/p + \log p)$, with p the number of cores [Martin and Cundy, 2018].

2.3 State Space models

The authors of [Gu et al., 2022a] have introduced a new model to capture long-range dependencies by simulating the State-Space Model (SSM). The model assumes the following:

$$h'(t) = Ah(t) + Bx(t), \quad (7a)$$

$$y(t) = Ch(t) + Dx(t) \quad (7b)$$

with $A \in \mathbb{R}^{D \times D}$, $B \in \mathbb{R}^{D \times L}$, $C \in \mathbb{R}^{S \times D}$ and $D \in \mathbb{R}^{S \times L}$, and with $t \in [0, T]$ a continuous time variable between the start time 0 and the final time T .

By discretizing the differential equations (7), it turns out that the model coincides with the Linear recurrent unit model suggested in Section 2.2. Indeed, the weight matrices can be rewritten in the discretized form with $t = \delta k$, δ being the step size of the discretization of the time t (proof in the Appendix A.1):

$$h_k = \tilde{A}h_{k-1} + \tilde{B}x_k, \quad (8a)$$

$$y_k = \tilde{C}h_k + \tilde{D}x_k \quad (8b)$$

with $\tilde{A} = \exp(\delta A)$, $\tilde{B} = (\exp(\delta A) - \text{Id}_N)A^{-1}B$, $\tilde{C} = C$ and $\tilde{D} = D$.

As the vanishing gradient problem remains from the model structure, the SSM aims at stabilizing the mixing layer by using a structured weight matrix A using the HiPPO (High-order Polynomial Projection Operators) theorem [Gu et al., 2020]:

$$A_{ij} = - \begin{cases} (2i+1)^{1/2}(2j+1)^{1/2} & \text{for } i > j, \\ i+1 & \text{for } i = j, \\ 0 & \text{for } i < j \end{cases}$$

The authors have further shown that the structure matrix A can be diagonalized [Gu et al., 2022b] by $A_d = -\frac{1}{2} + i\pi d$ (S4D-Lin) or $A_d = -\frac{1}{2} + i\frac{D}{\pi} \left(\frac{D}{2d+1} - 1 \right)$ (S4D-Inv), which leads then to the same settings as the LRUs.

Recently, the authors introduced a new model, Mamba [Gu and Dao, 2024], which adapts the weight matrices B , C , and the step size δ as a function to each input token x_k . The temporal adaptation of parameters allows the model to filter out useless information in the sequence of input tokens, leading to competitive results on complex tasks compared to state-of-the-art models.

2.4 Transformers

The transformers [Vaswani et al., 2017] aim to enhance the performance by computing relations between pairs of existing tokens via the so-called self-attention matrix. Instead of mixing the information from the previous tokens via the hidden state, the method learns the self-attention matrix and uses the feed-forward network to compute the probabilities of the output.

At first, the input data $\mathbf{x} \in \mathbb{R}^{N \times L}$ is projected to a high-dimensional space of dimension E by the embedding layer. Then, the positional encoding layer transforms the input data to save the positional information. The encoding layer adds the following information to the embedded input:

$$\text{PE}_{(\text{pos}, 2i)} = \sin \left(\frac{\text{pos}}{10000^{2i/E}} \right) \quad (9a)$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos \left(\frac{\text{pos}}{10000^{2i/E}} \right) \quad (9b)$$

where pos is the position of the token and i is the dimension of the embedded token. Storing the positional information of the tokens is crucial, as the self-attention matrix would evaluate the relations between the pairs of tokens without considering their orders otherwise. Let us define the output of the positional encoding $\tilde{\mathbf{x}} \in \mathbb{R}^{N \times E}$.

Then, to compute the self-attention matrix, the model learns three parameters: the query matrix $\mathbf{W}_Q \in \mathbb{R}^{D \times N}$, the key matrix $\mathbf{W}_K \in \mathbb{R}^{D \times N}$, and the value matrix $\mathbf{W}_V \in \mathbb{R}^{D \times N}$. The computation can be summarized as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{D}} \right) V \quad (10)$$

with $Q = \mathbf{W}_Q \tilde{\mathbf{x}}$, $K = \mathbf{W}_K \tilde{\mathbf{x}}$, $V = \mathbf{W}_V \tilde{\mathbf{x}}$ the projections of the embedded input $\tilde{\mathbf{x}}$. The performance is improved using several heads computed in parallel with multiple cores. Having multiple heads allows the capture of different patterns of the input tokens as each head uses different weight matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$. The self-attention matrices computed from H heads are concatenated to one final self-attention matrix. This matrix is then passed into a feed-forward neural network to fit the output. These feed-forward layers contain non-linear activation functions and can learn complex representations.

A simplified formula of the model is the following [Yun et al., 2020]:

$$\text{Attention}(\tilde{\mathbf{x}}) = \tilde{\mathbf{x}} + \sum_{i=1}^H \mathbf{W}_O^i \mathbf{W}_V^i \tilde{\mathbf{x}} \cdot \sigma[(\mathbf{W}_K^i \tilde{\mathbf{x}})^T \mathbf{W}_Q^i \tilde{\mathbf{x}}] \text{ with } H \text{ the number of heads.} \quad (11a)$$

$$\text{FF}(\text{Attention}(\tilde{\mathbf{x}})) = \text{Attention}(\tilde{\mathbf{x}}) + \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \text{Attention}(\tilde{\mathbf{x}}) + \mathbf{b}_1 \mathbf{1}_n^T) + \mathbf{b}_2 \mathbf{1}_n^T \quad (11b)$$

As the self-attention matrix computes the relations between all existing pairs of tokens, it is known that the model can capture well long-range dependencies [Zimerman and Wolf, 2023]. However, considering each token pair would lead to a significant increase in the computation cost. Indeed, the cost becomes $O(N^2)$, and the memory cost becomes $O(HN^2)$ with H heads.

To solve the need for this massive computation cost, the authors of [Lee-Thorp et al., 2022] have developed a model called FNet, which uses a Fourier mixing layer instead of self-attention layers. The latter layer applies a two-dimensional Discrete Fourier Transform on $\tilde{\mathbf{x}} \in \mathbb{R}^{N \times E}$.

$$\text{Fourier}(\tilde{\mathbf{x}}) = \Re(\mathcal{F}_N(\mathcal{F}_E(\tilde{\mathbf{x}}))) \quad (12)$$

which can be written as:

$$\text{Fourier}(\tilde{\mathbf{x}})_{n,e} = \Re \left(\sum_{k=0}^{N-1} \sum_{j=0}^{E-1} \tilde{x}_{k,j} \exp \left(-2\pi i \left(\frac{kn}{N} + \frac{je}{E} \right) \right) \right). \quad (13)$$

The feed-forward neural networks can be computed similarly to (11b):

$$\text{FF}(\text{Fourier}(\tilde{\mathbf{x}})) = \text{Fourier}(\tilde{\mathbf{x}}) + \mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \text{Fourier}(\tilde{\mathbf{x}}) + \mathbf{b}_1 \mathbf{1}_n^T) + \mathbf{b}_2 \mathbf{1}_n^T \quad (14)$$

Discrete Fourier Transform replaces the computation of self-attention matrix and can be computed in $O(N \log N)$ using Fast Fourier Transform algorithms [Cooley and Tukey, 1965].

2.5 A meta-model

From the presented models in previous sections, some common features have been shared across their architectures (except non-linear RNNs). Notably, the main framework of the models is built upon two stages: first, after embedding the input data, the model incorporates a linear mixing layer or a self-attention layer to transform the input data adapted to capture long-range dependencies. Then, the models add a feed-forward neural network to fit the transformed data to the output to approximate the non-linear complex target function. The two stages can be translated as follows:

$$\hat{x}_k = \sum_{l=1}^N \tilde{A}_{k,l} x_l \quad (15a)$$

$$\tilde{\mathbf{x}} = \text{FF}(\hat{\mathbf{x}}) \quad (15b)$$

with $x_l \in \mathbf{x}$. The identity of the different models come from the definition of the matrix $\tilde{A}_{k,l}$ in equation (15a), dependent from k , the position of the token, and l the iterating variable over the whole token sequence. First, we can regroup the linear RNN-based models (LRU and SSM) as they share a similar structure. The matrix $\tilde{A}_{k,l}^{\text{IRNN}}$ of linear RNN models is defined as follows:

$$\tilde{A}_{k,l}^{\text{IRNN}} = \mathbb{1}_{l \leq k-1} C A^{k-1-l} B + \mathbb{1}_{l=k} D$$

The matrix $\tilde{A}_k^{\text{Transformers}}$ for transformers can be defined as follows when omitting σ :

$$\tilde{A}_{k,l}^{\text{Transformers}} = \mathbb{1}_{k=l} Id_N + W_O W_V (x_k^T W_K^T W_Q).$$

FNet uses the Discrete Fourier Transform on both dimensions of the input data $\mathbf{x}$. We have:

$$\tilde{A}_{k,l}^{\text{FNet}} = \mathbf{W}_1 \mathbf{M}$$

where with $\mathbf{M} \in \mathbb{R}^{N \times D}$ a matrix defined as $\mathbf{M}_{i,j} = \begin{cases} 1 & \text{if } i = l, \\ 0 & \text{else} \end{cases}$ with $i \in [1, N]$ and $j \in [1, D]$. The feed-forward neural network of FNet can be computed as follows:

$$\text{FF}(\hat{\mathbf{x}}) = \sigma(\hat{\mathbf{x}} \mathbf{W}_2 \mathbf{W}_3) \mathbf{W}_4,$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are the Fourier Transform matrix of $\mathbf{X} \in \mathbb{R}^{N \times D}$ with respect to the length of the sequence N and the hidden dimension D . $\mathbf{W}_3$ and $\mathbf{W}_4$ are the weight matrices of the feed-forward neural network, with σ the activation function in the hidden layer. One detail to notice is that the matrix $\mathbf{W}_2$, which is the Fourier transform matrix of $\hat{\mathbf{x}}$ along the dimension D , can be detached and be computed in the feed-forward neural network step. Among the different representations, the matrix $\tilde{A}_k^{\text{Transformers}}$ is the only one dependent from $\hat{\mathbf{x}}$.

3 Experimental results

Focusing on LRU, we have reproduced the results from [Orvieto et al., 2023]. Then, by differing the hyperparameters and the model architecture, we measured their impact on the performance and the tendency during the training process.

3.1 Main framework of the model

As described in the original paper [Orvieto et al., 2023], the parameterization has a significant impact on the performance of the model, as shown by the authors with the thorough construction of the final model in equation 7a. Furthermore, many learnable parameters are added from the MLP block and a stack of hyperparameters to tune. The report will focus mainly on the impact of hyperparameters outside of the LRU block and see how they can improve the training process.

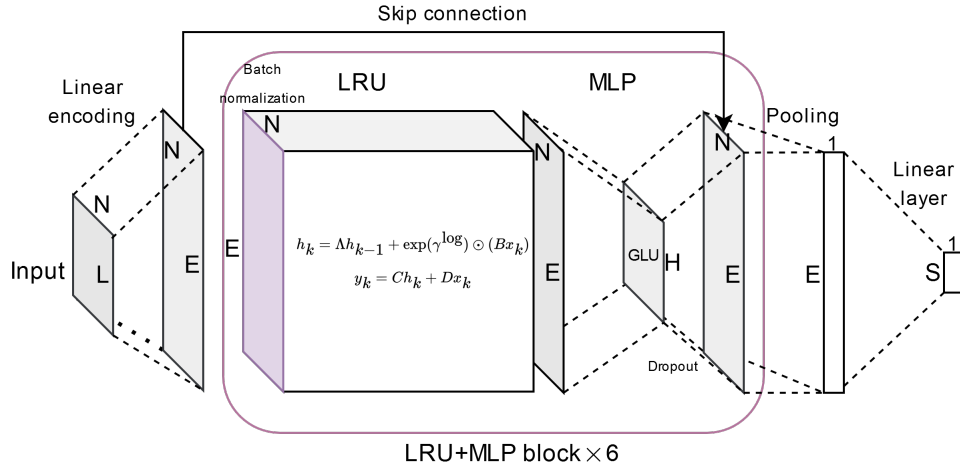


Figure 3.1: Scheme of LRU used in the experiments in Section 3.2.

Figure 3.1 is the scheme of the main framework used in the experiments in this report. Although most of the architecture has been preserved from the original paper [Orvieto et al., 2023], we have

changed some of the details to enhance the accuracy as we found that they were not giving the same results. For instance, the default learning rate is set at 0.001 instead of 0.004 in our setting since a more significant learning rate triggered a sudden drop in the accuracy and a divergence of the loss in the middle of the training process. Also, the skip connection is within the LRU and MLP block in the original paper, but as shown later, this structure hinders the speed of the test accuracy from increasing while overfitting the data. Instead, We have chosen to take the initial copy of the direct output from the linear encoding layer and add it at the end of each LRU and MLP block, significantly increasing the performance.

To go further, we have conducted several runs using different values of the hyperparameters or model structures to see their effects while training the model. First, we tested the model with varying depths of LRU and MLP blocks, going from one to seven. Then, we used various numbers of neurons in the hidden layer of the MLP, going through the Gated Linear Unit (GLU) activation function [Dauphin et al., 2017]. Also, we have tested with different learning rates as they are known to play a significant role in the training process of deep neural networks [Witten et al., 2017]. Then, we conducted an ablation study by removing several parts of the model structure, such as batch norm, dropout, learning rate factor, or weight decay. Furthermore, we tried to use the skip connection independently for each block, like on the original paper, or replace the pooling layer with a trainable linear layer. Last, we have experimented with the learning rate and the standard deviation of the sampling of the initial weight matrices of the MLP and the linear transformations suggested from [Chizat and Netrapalli, 2024].

3.2 Results of the experiments

In this section, we present the results of the empirical research mentioned previously. All of the experiments are tested on a sequential CIFAR-10 dataset, maintaining the color information of the input images as done in [Orvieto et al., 2023]. The experiments are run using Python and primarily from the library `Flax` [Heek et al., 2024]. The main framework of the code for running the experiments can be found in <https://github.com/seorimpark/LRU>. The experiments were all run on an A100 GPU.

The variables and hyperparameters of the default setting showed in Figure 3.1 were fixed by default as follows:

- According to the size of the sequential CIFAR-10 dataset, we have the total number of tokens N equal to 1024 (as the images are of both width and height 32 pixels), the length of one token L equal to 3, the RGB color information from the image pixels.
- The encoding size E preserved throughout the training model is fixed to 512.
- The output $\mathbf{y}$ is a vector of size 10, equal to the number of classes of the images.
- The batch size B of the input data is set to 50.
- The number of training steps is fixed at 30,000 due to the limitations in the computation resources. The training set of CIFAR-10 contains 50,000 images, so the experiments were run for 30 epochs.

Following the setting from the original paper, we used a warm-up starting from 10^{-7} until reaching the desired maximum learning rate for the first 10% of the training steps, then we used the cosine annealing method, bringing down back to 10^{-7} towards the end of the training.

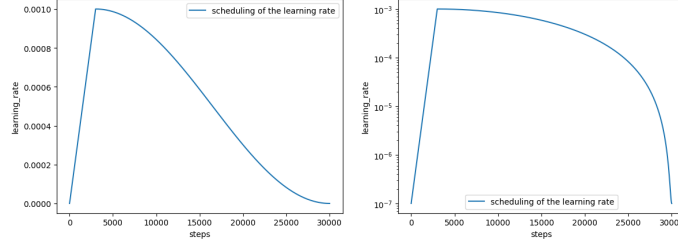


Figure 3.2: To the left is a graph depicting the tendency of the learning rate with linear warm-up until the first 10% of 30,000 steps, then using cosine annealing for the rest of the training. The maximum learning rate is attained at the defined learning rate (here at 0.001), and the minimum learning rate is fixed to 10^{-7} , attained at the start and end of the run. To the right is the same graph with a log-scaled y-axis.

Figure 3.2 shows the evolution of the learning rate as a function of the experiments’ steps. The plots show that the learning rate reaches its maximum at step 3,000. The selected optimizer algorithm is AdamW from [Loshchilov and Hutter, 2019].

Furthermore, it is necessary to decide upon a default model setting to perform the experiments to focus on the impact of a single parameter. For the reliability of the results, each experiment except the removal of the batch normalization and the layer-wise parameterization are run five times with randomized initial parameters to have a sketch of the confidence interval of the values. The following list is the parameters to be fixed by default, then changed one by one:

- The depth of the LRU and MLP blocks is six by default, placed sequentially one after the other before reaching the pooling layer.
- The dimension of the hidden state D of the LRU block is fixed to 384.
- The number of neurons H in the hidden layer of the MLP block is set to 384.
- The maximum learning rate is fixed at 0.001.
- The training parameters in the LRU block (ν , θ , and B) are set to have a smaller learning rate than the other parameters with a learning rate factor equal to 0.25.
- The dropout of the hidden layer of the MLP is set to 0.1.
- The training parameters except ν , θ and B have a weight decay equal to 0.05.
- The pooling layer takes the mean of the values across the tokens, and then a linear layer is applied to get the final output.

To compare our results with the one from the original paper, we have run the training for 180,000 steps with the parameters fixed from above and following the scheme from Figure 3.1, and we were able to achieve 87% accuracy on test data by the end of the training. The accuracy achieved in the paper was 89%.

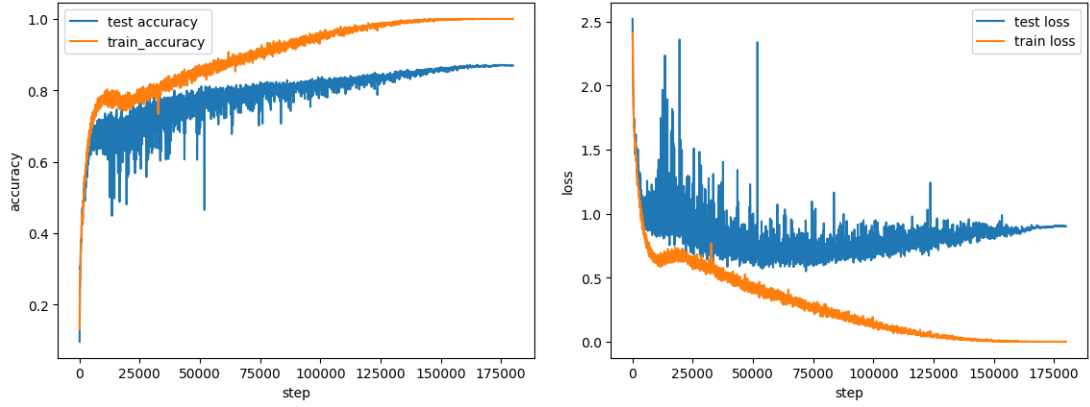


Figure 3.3: The plots of the results of the training with selected default parameters, for 180,000 steps. To the left is the train and test accuracy plot, and to the right is the train and test loss plot.

Figure 3.3 shows the plot of the accuracy and loss during the training. While running the experiment, we could spot that the overfitting problem heavily influenced the final test accuracy. The training accuracy attained 100% at step 160,000, and the test accuracy remained at 87% percent until the end of the training. This tendency can be observed in the loss plot, as the training loss decreases almost steadily during the training, while the test loss tends to increase by the end.

During the analysis of the results, we will focus on three points:

- The test accuracy by the end of the training.
- The difference between the train and the test accuracy to estimate the degree of overfitting during training. As the resource is restrained, most experiments do not attain the 100% training accuracy. It is important to see how much the model is overfitted to estimate how far the test accuracy can increase.
- The speed of convergence of the test accuracy and its relative order compared to the other models.

3.2.1 Depth

First, the depth of the LRU and MLP blocks is important in training the model, notably in computation time, as it heavily influences the number of operations needed per training step.

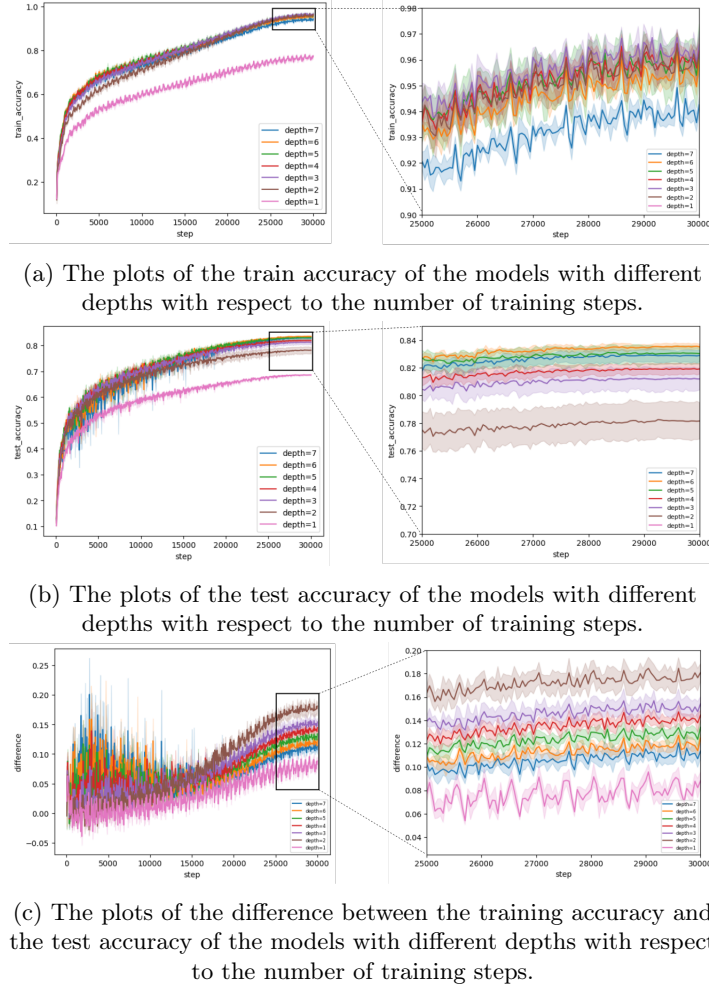
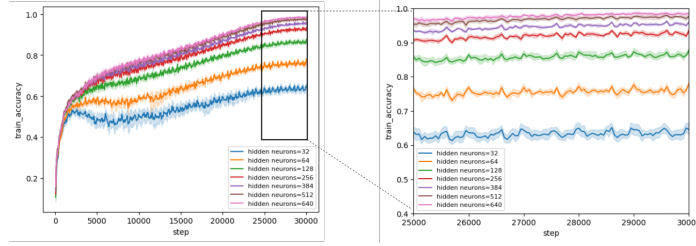


Figure 3.4: The plots of the accuracy of the models with different depths from one to seven. For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting.

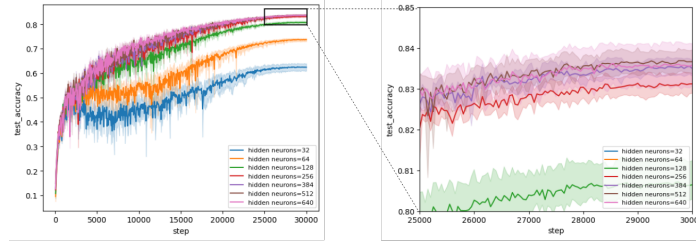
In Figure 3.4, the most distinctive result from the plots is that the case with depth equal to one is distinguished from the rest of the models. Indeed, the test accuracy, the training accuracy, and the difference between the two accuracies are far lower than the rest. The results show that apart from the mentioned case, the rest of the models have more or less a similar tendency, especially in the training accuracy. The width of the confidence interval overlaps between each of them towards the end of the training. Also, the confidence interval of the test accuracy for the model at depths five, six, and seven overlap, which means that it is possible to have a good result by running with only five LRU and MLP blocks. The plot of the difference between the accuracies shows that apart from depth one, increasing the depth resulted in better generalization as there is less overfitting.

3.2.2 Hidden dimension of the MLP

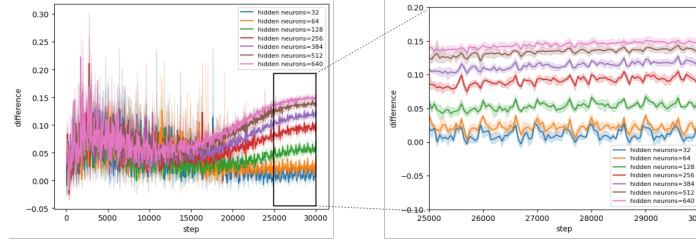
A second hyperparameter to consider is the number of neurons within the hidden layer for each MLP block. The theoretical insights from [Orvieto et al., 2024] state that there exists a lower bound of the number of hidden neurons needed to fit the non-linearity of the target function f , which means increasing the number of neurons after this bound does not necessarily lead to a better result. Our goal is to explore the empirical results of this statement by testing on the CIFAR10 dataset.



(a) The plots of the training accuracy of the models with different numbers of neurons in the hidden layer of the MLP with respect to the number of training steps.



(b) The plots of the test accuracy of the models with different numbers of neurons in the hidden layer of the MLP with respect to the number of training steps.



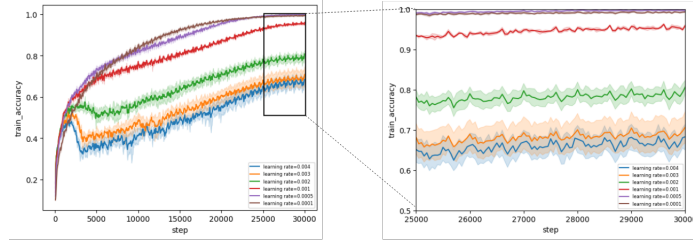
(c) The plots of the difference between the training accuracy and the test accuracy of the models with various numbers of neurons in the hidden layer of the MLP, with respect to the number of training steps.

Figure 3.5: The plots of the accuracy of the models with different numbers of neurons in the hidden layer of the MLP in each LRU and MLP block (32,64,128,384,512,640). For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting.

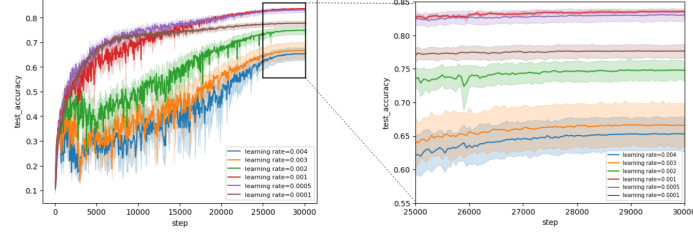
According to Figure 3.5, the number of neurons becomes essential when considering the overfitting problem. While the test accuracy remains similar for numbers greater than 256, the bigger the number is, the faster the training accuracy converges to the unity. A smaller number of neurons leads to a slower convergence of both train and test accuracy and performs poorly in general, as the graph stagnates towards the end of the training. In addition, more neurons do not result in better test accuracy by the end of the training, although it helps stabilize the graph's tendency. The plots of the difference in accuracies show that globally, models with more neurons are more prone to overfitting globally.

3.2.3 Learning rate

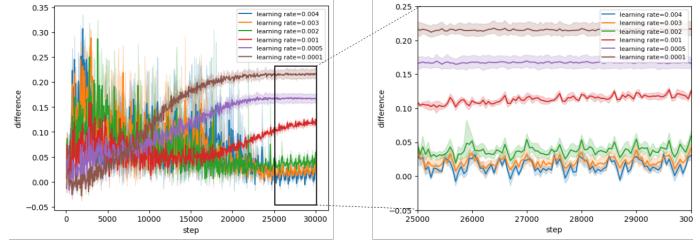
The learning rate is an essential factor when training the model, as it plays a significant role in approximating the results. A lower learning rate will result in slower convergence and overfitting [Smith, 2018]. A more significant learning rate could induce a divergence and an explosion of loss during the training. This section focuses on testing the model with different learning rates.



(a) The plots of the train accuracy of the models with different learning rates with respect to the number of training steps.



(b) The plots of the test accuracy of the models with different learning rates with respect to the number of training steps.



(c) The plots of the difference between the training accuracy and the test accuracy of the models with different learning rates with respect to the number of training steps.

Figure 3.6: The models' accuracy plots with different learning rates (0.0001, 0.0005, 0.001, 0.002, 0.003, 0.004). For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting.

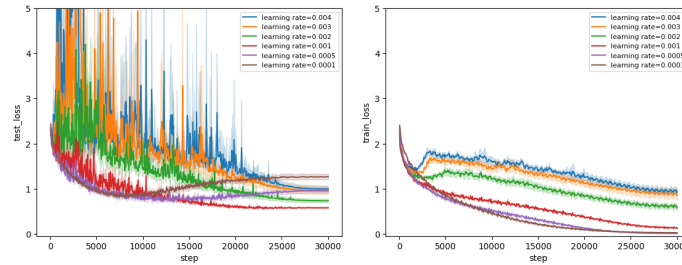


Figure 3.7: The plots of the loss of the models with different learning rates with respect to the number of training steps. To the left is the plot of the test loss and to the right is the plot of the training loss.

Figure 3.6 shows that the tendency of the graphs varies a lot concerning the learning rate. The plots of the loss in Figure 3.7 are added to show the divergence when the learning rate is more significant

than 0.002, although the loss comes back to the track after a few explosions. Tuning the optimal learning rate is essential since the training accuracy attained 100% in the 30,000 steps while the test accuracy stopped at 78% when the learning rate was equal to 0.0001. The plots of the test accuracy and the training accuracy show that for learning rates more significant than 0.002, there were a few sudden drops in the middle of the training, hindering the convergence and, hence, the performance by the end. The drop occurred around step 3,000, where the actual learning rate is at the maximum according to Figure 3.2. On the one hand, the difference in the accuracies shows that only minor overfitting is present for significant learning rates, but the result might be due to the slow convergence from the drops. On the other hand, the plots of the loss show that while the convergence of the graph with more significant learning rates is unstable but going downwards globally, the ones with lower learning rates tend to have an increased test loss by the end, showing that overfitting is present.

3.2.4 Ablation studies

The model incorporates several methods to boost performance, although it is unknown how influential they are compared to the one from the core model structure, which is the LRU and MLP block. It is therefore necessary to test the effects of each method by doing an ablation study on the existing techniques, such as batch normalization, dropouts, weight decays, and the learning rate parameterization restricted for LRU blocks.

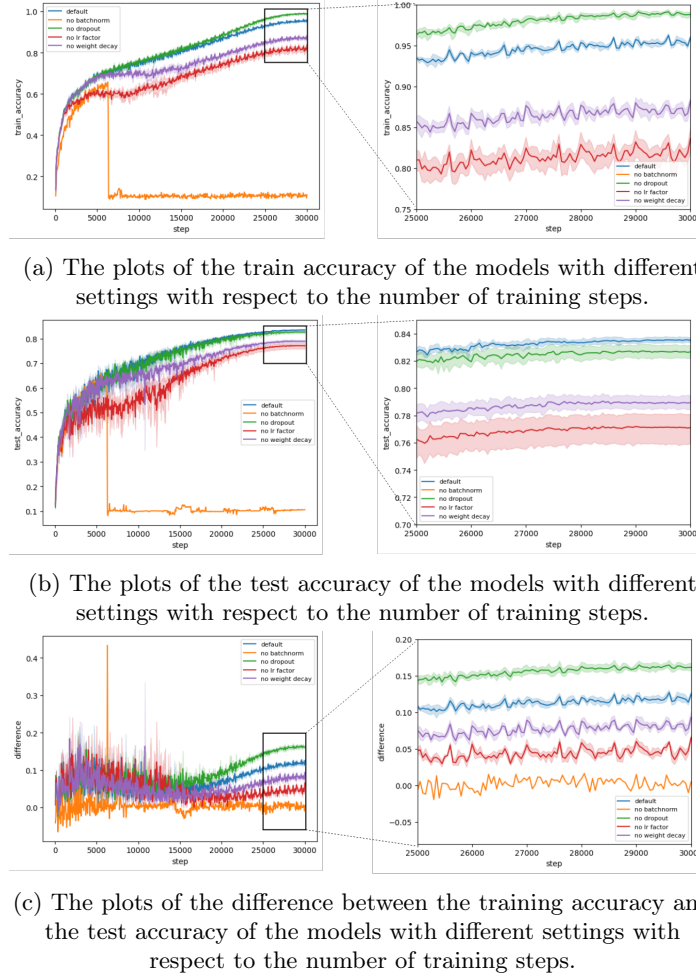


Figure 3.8: The plots of the accuracy of the models with different settings (removal of batch normalization, dropout, learning rate factor, and weight decay). For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting.

Figure 3.8 shows that removing parts of the model settings has a more or less significant effect on the performance. First, removing the batch normalization at the beginning of each LRU and MLP block induces a divergence in the train and test accuracy. This result shows that the normalization step is essential, especially when paired with skip connections, as it adds the initial data to the transformed data at each stage. Second, removing the dropout created more overfitting, as shown in the plot of the differences between the two accuracies. Although the training accuracy converged faster, the model with dropouts performed better on the test accuracy. Third, removing the learning rate factor for the LRU block significantly reduced the performance, and the convergence of the training accuracy became slower and unstable, as shown in the result in Figure 3.6 with significant learning rates. Last, the results show that weight decay is essential to boost convergence speed.

3.2.5 Change in the structure

In addition to the ablation studies, we have tested several settings to check the original paper’s results.

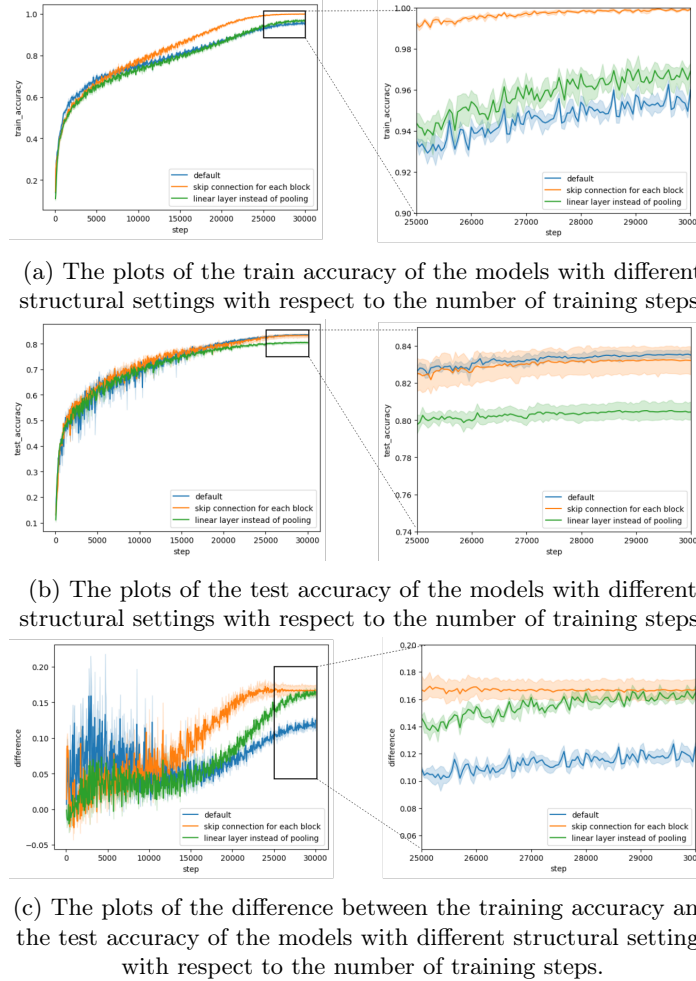


Figure 3.9: The plots of the accuracy of the models with different structural settings (adding a skip connection for each block, adding a linear layer instead of the pooling layer). For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting.

The plots in Figure 3.9 show that the pooling layer gave a boost in the performance compared to the case with the linear layer instead, but isolating the skip connections for each LRU and MLP block

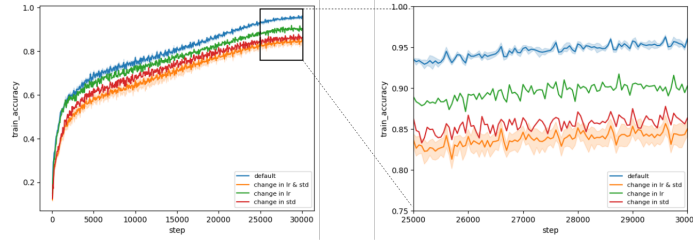
did not help in improving the test accuracy. The pooling layer retained the convergence speed of the training accuracy and gave a boost to the convergence speed of the validation accuracy. Replacing the pooling layer with a linear layer is, therefore, not beneficial. Isolated skip connections induced serious overfitting problems compared to the skip connection from the initial setting.

3.2.6 Layer-wise parameterization

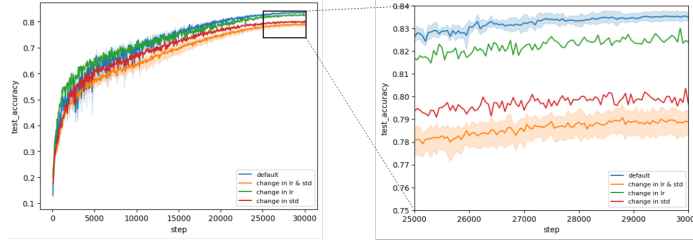
In addition to the model structure change, we tried optimizing the parameters layer-wise, as shown in [Chizat and Netrapalli, 2024]. The paper presents an efficient approach to parameterizing the learning rate and the scale of the initial values from the weight matrices of linear layers in models incorporating MLPs. Let us set d as the number of dimensions of the input layer and m as the number of neurons in the hidden layer. k is the number of dimensions of the output layer, and L is the number of hidden layers. As we are in a setting with one hidden-layered MLP, we have three categories of layers to parameterize:

- The "input" layer: the learning rate factor is m/d and the scale of the initial weight matrix is set as $1/\sqrt{d}$.
- The "hidden" layer: the learning rate factor is set as 1, and the scale of the initial weight matrix is set as $2/\sqrt{m}$.
- The "output" layer: the learning rate factor is set as k/m , and the scale of the initial weight matrix is set as $\sqrt{k}/m$.

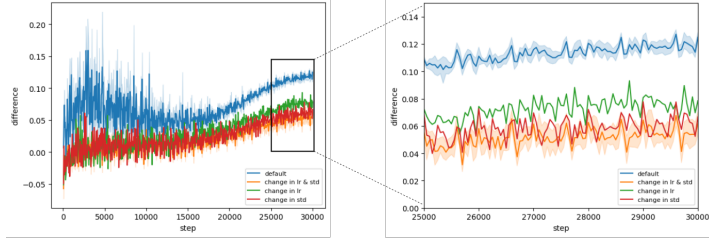
The model sets the linear encoding layer as the "input layer," the first layer of the MLP block is set as the "hidden" layer, and the second MLP layer, as well as the final linear projection to the output, is set as the "output" layer. We have tested three cases: incorporating only the learning rate factors or the scale and both of them. Due to the limited resources, we could not run all of the experiments five times; the only case was when we parameterized both learning rates, and the initial scale was run multiple times.



(a) The plots of the training accuracy of the models with different layer-wise parameterization settings with respect to the number of training steps.



(b) The plots of the test accuracy of the models with different layer-wise parameterization settings with respect to the number of training steps.



(c) The plots of the difference between the training accuracy and the test accuracy of the models with different layer-wise parameterization settings with respect to the number of training steps.

Figure 3.10: The plots of the accuracy of the models with layer-wise parameterization (initial scale, learning rate factor, and both). For each row, to the left are the entire plots, and to the right are the zoomed plots from step 25000 to 30000. The colored areas are the confidence interval of 95%, computed by the results of five runs for each setting (except for the parameterization of learning rates or the scale only).

Figure 3.10 shows that the performance is globally worse when using layer-wise parameterization. However, we observed that the latter guaranteed a stable convergence of the test accuracy compared to the default setting. Setting the new initial scales of the weight matrices showed a significant decrease in both accuracies. Tuning only the learning rate factors solved the overfitting problem while maintaining a reasonable loss in the test accuracy compared to the default setting. Tuning both parameters resulted in lower test accuracy towards the end, but as shown in the plot of the difference of accuracies, this change helped dealing with the overfitting problem. We have tested this setting with step size 180,000, and the test accuracy reached 86.2%, while the training accuracy reached 100% at around step 170,000 (plot in the Appendix B.1).

3.3 Summary of the results

In this section, we present the results of the training with the different settings presented in the previous sections. The values of accuracy, loss, and running time are the averages measured over the five runs for each setting.

Depth	Number of training parameters	Running time (min)	Train accuracy	Test accuracy
1	1,092,106	20	0.77396	0.68582
2	2,177,034	35	0.96237	0.78150
3	3,261,962	50	0.96596	0.81182
4	4,346,890	67	0.96359	0.81910
5	5,431,818	82	0.96425	0.83020
6	6,516,746	104	0.95992	0.83502
7	7,601,674	120	0.94253	0.82886

Table 1: Results of the training with different depths

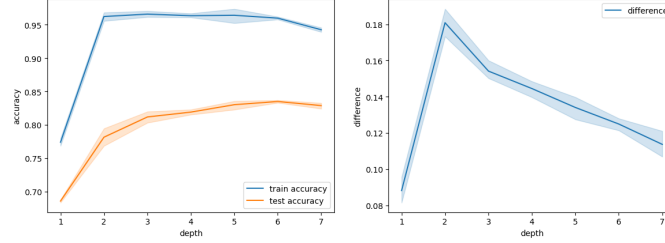


Figure 3.11: The plot of the training accuracy and the test accuracy (left) and the plot of the difference between the two latter (right) at step 30,000 with respect to the depth.

The depth of the model has a significant impact on the running time as most of the trainable parameters are in the LRU and MLP block. The test accuracy for depth five is not much different from the one with depth six, which implies that using the model with depth five can be a better choice when considering the time and the computational cost induced by the complexity of the model. Figure 3.11 shows that the test accuracy indeed stagnates from depth five. In contrast, the plot of the difference in the accuracies shows a linear fit, inversely proportional with respect to the depth, except in the case of depth one.

number of neurons in the hidden layer	Number of training parameters	Running time (min)	Train accuracy	Test accuracy
32	4,892,618	97	0.64531	0.62362
64	5,040,266	98	0.77984	0.73634
128	5,335,562	98	0.87633	0.80644
256	5,926,154	101	0.93429	0.83122
384	6,516,746	104	0.95992	0.83502
512	7,107,338	105	0.97665	0.83666
640	7,697,930	107	0.98612	0.83578

Table 2: Results of training with different numbers of neurons in the hidden layer of the MLP.

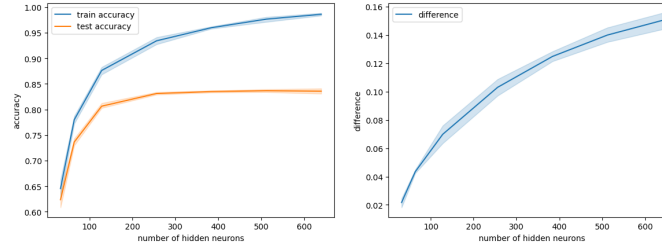


Figure 3.12: The plot of the training accuracy and the test accuracy (left) and the plot of the difference between the two latter (right) at step 30,000 with respect to the number of hidden neurons.

Before all, the number of neurons had a limited effect on the running time as the variation in the number of training parameters was minimal compared to the case of varying the depth. The test accuracy by the end of the training shows that adding the number of neurons does not necessarily give better performance, and setting it to 384 is the best choice when considering the performance and the overfitting problem. The results align with the theoretical insight from Section 2.2 according to Figure 3.12: the number of hidden neurons did not significantly alter the validation accuracy after 256 neurons. The overfitting problem worsens as the number of neurons grows, and the final difference increases almost linearly.

Learning rate	test loss	Train accuracy	Test accuracy
0.0001	1.27554	0.99331	0.77648
0.0005	1.04665	0.99641	0.83016
0.001	0.57934	0.95992	0.83502
0.002	0.66664	0.80702	0.74782
0.003	0.82575	0.70743	0.66578
0.004	1.09954	0.68335	0.65286

Table 3: Results of the training with different learning rates.

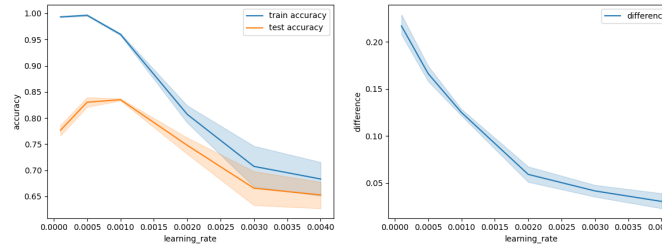


Figure 3.13: The plot of the training accuracy and the test accuracy (left) and the plot of the difference between the two latter (right) at step 30,000 with respect to the learning rate.

The results from Table 3 show that having a lower learning rate (0.0001) performs almost as poorly as the case when having a significant learning rate with sudden drops (0.002). The test loss is significantly larger when the learning rate is too small or too big. The tendency of the validation accuracy as a function of the learning rate of the plot in Figure 3.13 shows a global maximum is attained between 0.0005 and 0.001.

Model	Number of training parameters	Running time (min)	Train accuracy	Test accuracy
default	6,516,746	104	0.95992	0.83502
no batchnorm	6,510,602	93	0.1102	0.1052
no dropout	6,516,746	104	0.98824	0.82668
no lr factor	6,516,746	105	0.83641	0.77108
no weight decay	6,516,746	103	0.88245	0.78938

Table 4: Results of the training from the ablation studies

The number of trainable parameters did not change except for the case without the batch normalization, so the running time remained similar. The batch normalization played a significant role in the training, as we observed that the training and the test accuracy could not be restored until the end. The removal of the learning rate factor and the weight decay had a more significant impact on the performance than the case without the dropout.

Model	Number of training parameters	Running time	Train accuracy	Test accuracy
default	6,516,746	104	0.9599	0.83502
skip connection for each block	6,516,746	104	0.99935	0.83214
Linear layer instead of pooling	6,517,770	104	0.96906	0.80434

Table 5: Results of the training from the change of the model structure.

Table 5 shows that isolated skip connections induced overfitting when comparing the training accuracy. Using linear layers instead of the pooling layer sped up the convergence of the training accuracy.

Model	Train accuracy	Test accuracy
default	0.9599	0.83502
Learning rate and standard deviation	0.85012	0.78870
Learning rate	0.90245	0.824
Standard deviation	0.86327	0.79670

Table 6: Results of the training from the layer-wise parameterization of the model.

Table 6 presents the results of the training with layer-parameterization. Although we could run the experiments with the tuning of the standard deviation or the learning rate only once, the results show that globally, the layer-wise parameterization following [Chizat and Netrapalli, 2024] helped mitigate the overfitting problem. However, the training and validation accuracy convergence speed were slower than in the standard setting.

4 Discussion

This work provides insight into the different existing deep neural network models, such as Recurrent Neural Networks (RNNs), Linear Recurrent Units (LRUs), State-space models (SSMs), and transformers, and how they balance efficiency and performance. Within them, LRUs have shown unexpectedly good performance over complex tasks by pairing up with an MLP and increasing its depth [Orvieto et al., 2023]. The theoretical analysis from [Orvieto et al., 2024] has shown the capacity of LRU to perform lossless compression while fitting well to non-linear complex target functions via the combination with MLPs.

Empirical evaluations of LRUs validate the previous findings by exhibiting the impact of architectural configurations and hyperparameter tuning on performance. For instance, deeper LRU and MLP blocks improve generalization while mitigating overfitting but taking a lot of compute power, whereas shallower LRU and MLP blocks are underperforming, although they are more efficient. This phenomenon highlights the importance of balancing the model complexity with resource constraints. Furthermore, the experiments on the hidden dimensions in the MLP blocks support the statements made from the theoretical insights about LRUs, as they reveal that the performance stagnates beyond a certain threshold of neurons.

The results also highlight the importance of incorporating optimization techniques such as batch normalization, dropout, and weight decay, which helped stabilize training and address overfitting. Ablation studies confirmed that those components are necessary for better performance, with batch normalization playing a significant role. In addition, modifications to the model structure, such as modifying skip connection configurations or replacing pooling layers with linear transformations, provided insights into effective design choices.

Despite their computational advantages, LRUs show limited performance. The maximum validation accuracy of 87% on the sequential CIFAR-10 dataset shows that they still lack performance compared to state-of-the-art models like transformers-based models that reached an accuracy beyond 99% [Dosovitskiy et al., 2021]. Nonetheless, the reduced complexity and scalability of LRU show that the latter can be a good substitute in resource-constrained settings. Also, the side-by-side comparison of the results from our experiments with the results obtained from the source paper is subtle due to restrained resources. More reliable results could have been obtained by optimization of the code or by having access to more computational resources.

Overall, this study gives clues on the correct construction of the architecture of deep learning models used in practice and the appropriate parameterization to maximize performance, starting from the case of LRUs. Future works can focus on extending these findings by integrating LRUs with other deep learning techniques and by applying the model to larger datasets or more intricate tasks. Such exploration could aid in finding a new model that fills the performance gap while retaining computational efficiency.

References

- Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew Ballard, Joshua Bambrick, Sebastian Bodenstein, David Evans, Chia-Chun Hung, Michael O’Neill, David Reiman, Kathryn Tunyasuvunakool, Cervantes Wu, Akvilė Žemgulytė, Eirini Arvaniti, and John Jumper. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630:493–500, 05 2024. doi: 10.1038/s41586-024-07487-w.
- Gaurang Bansal, Vinay Chamola, Amir Hussain, Mohsen Guizani, and Dusit Niyato. Transforming conversations with ai—a comprehensive study of chatgpt. *Cognitive Computation*, 16:1–24, 01 2024. doi: 10.1007/s12559-023-10236-2.
- A.R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, 39(3):930–945, 1993. doi: 10.1109/18.256500.
- Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. doi: 10.1109/TPAMI.2013.50.
- Guy E. Blelloch. Prefix sums and their applications, 1990. URL <https://api.semanticscholar.org/CorpusID:60459178>.
- Lénaïc Chizat and Praneeth Netrapalli. The feature speed formula: a flexible approach to scale hyperparameters of deep neural networks, 2024. URL <https://arxiv.org/abs/2311.18718>.
- James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Math. Comput.*, 19(90):297–301, 1965. doi: 10.2307/2003354.
- Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks, 2017. URL <https://arxiv.org/abs/1612.08083>.

- DeepMind, 2018. URL <https://deepmind.google/technologies/alphafold/>.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL <https://arxiv.org/abs/2010.11929>.
- Jean Ginibre. Statistical Ensembles of Complex, Quaternion and Real Matrices. *J. Math. Phys.*, 6: 440–449, 1965. doi: 10.1063/1.1704292.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Re. Hippo: Recurrent memory with optimal polynomial projections, 2020. URL <https://arxiv.org/abs/2008.07669>.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces, 2022a. URL <https://arxiv.org/abs/2111.00396>.
- Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. On the parameterization and initialization of diagonal state space models, 2022b. URL <https://arxiv.org/abs/2206.11893>.
- Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2024. URL <http://github.com/google/flax>.
- Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6:107–116, 04 1998. doi: 10.1142/S0218488598000094.
- Daniel Justus, John Brennan, Stephen Bonner, and Andrew Stephen McGough. Predicting the computational cost of deep learning models, 2018. URL <https://arxiv.org/abs/1811.11880>.
- Joe Kilian and Hava T. Siegelmann. The dynamic universality of sigmoidal neural networks. *Information and Computation*, 128(1):48–56, 1996. ISSN 0890-5401. doi: <https://doi.org/10.1006/inco.1996.0062>. URL <https://www.sciencedirect.com/science/article/pii/S0890540196900620>.
- Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. Deep learning. *Nature*, 521:436–444, 2015. doi: <https://doi.org/10.1038/nature14539>.
- James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontanon. Fnet: Mixing tokens with fourier transforms, 2022. URL <https://arxiv.org/abs/2105.03824>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Eric Martin and Chris Cundy. Parallelizing linear recurrent neural nets over sequence length, 2018. URL <https://arxiv.org/abs/1709.04057>.
- Namkee Oh, Gyu-Seong Choi, and Woo Yong Lee. Chatgpt goes to the operating room: evaluating gpt-4 performance and its potential in surgical education and training in the era of large language models. *Annals of Surgical Treatment and Research*, 104:269, 04 2023. doi: 10.4174/astr.2023.104.5.269.
- OpenAI, 2022. URL <https://chatgpt.com/>.
- Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences, 2023. URL <https://arxiv.org/abs/2303.06349>.
- Antonio Orvieto, Soham De, Caglar Gulcehre, Razvan Pascanu, and Samuel L. Smith. Universality of linear recurrences followed by non-linear projections: Finite-width guarantees and benefits of complex eigenvalues, 2024. URL <https://arxiv.org/abs/2307.11888>.

- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks, 2013. URL <https://arxiv.org/abs/1211.5063>.
- Lucas Høyberg Puvis de Chavannes, Mads Guldberg Kjeldgaard Kongsbak, Timmie Rantzau, and Leon Derczynski. Hyperparameter power impact in transformer language model training. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 96–118, Virtual, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.sustainlp-1.12. URL <https://aclanthology.org/2021.sustainlp-1.12/>.
- D. Rumelhart, G. Hinton, and R. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: <https://doi.org/10.1038/323533a0>.
- Leslie N. Smith. A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay, 2018. URL <https://arxiv.org/abs/1803.09820>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. Chapter 10 - deep learning. In Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal, editors, *Data Mining (Fourth Edition)*, pages 417–466. Morgan Kaufmann, fourth edition edition, 2017. ISBN 978-0-12-804291-5. doi: <https://doi.org/10.1016/B978-0-12-804291-5.00010-6>. URL <https://www.sciencedirect.com/science/article/pii/B9780128042915000106>.
- Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions?, 2020. URL <https://arxiv.org/abs/1912.10077>.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization, 2017. URL <https://arxiv.org/abs/1611.03530>.
- Itamar Zimmerman and Lior Wolf. On the long range abilities of transformers, 2023. URL <https://arxiv.org/abs/2311.16620>.

A Proofs

A.1 Proof of discretization in Section 2.3

Proof. Applying the Zero Order Hold (ZOH), we change the continuous time frame to discrete by letting $t = \Delta k$, where $k \in \mathbb{N}$ and $\Delta \in (0, 1)$ the step. Then value of $x(t)$ becomes $x(\Delta k)$ constant for $\Delta k < t < \Delta(k + 1)$. Then we have:

$$h'(t) = Ah(t) + Bu(\Delta k)$$

The solution of the differential equation is the sum of the homogeneous and the particular solution. The homogeneous solution of $h(\tau)$ is:

$$h(\tau) = \exp(A(\tau - \tau_0))h(\tau_0)$$

Taking $\tau = \Delta(k + 1)$ and $\tau_0 = \Delta k$, we get:

$$h(\Delta(k + 1)) = \exp(\Delta A)h(\Delta k).$$

Then, we want to find the particular solution of $h'(t) = Ah(t) + Bu(\Delta k)$. We have:

$$h(\Delta(k + 1)) = \exp(\Delta A)h(\Delta k) + \int_{\Delta k}^{\Delta(k+1)} \exp(A(\Delta(k + 1) - \theta))Bx(\Delta k)d\theta$$

If we do a change of variables $\mu = \Delta(k+1) - \theta$ and $d\mu = -d\theta$

$$h(\Delta(k+1)) = \exp(\Delta A)h(\Delta k) + x(\Delta k) \int_0^\Delta \exp(A\mu) B d\mu$$

$$h(\Delta(k+1)) = \exp(\Delta A)h(\Delta k) + (\exp(\Delta A) - I)A^{-1}Bx(\Delta k)$$

Changing the notations from $h(\Delta k)$ to h_k and from $x(\Delta k)$ to x_k , we obtain the following equations:

$$h_k = \exp(\Delta A)h_{k-1} + (\exp(\Delta A) - I)A^{-1}Bx_k, \quad (16a)$$

$$y_k = Ch_k + Dx_k \quad (16b)$$

Letting $\tilde{A} = \exp(\Delta A)$ and $\tilde{B} = (\exp(\Delta A) - I)A^{-1}B$ we get:

$$h_k = \tilde{A}h_{k-1} + \tilde{B}x_k, \quad (17a)$$

$$y_k = Ch_k + Dx_k \quad (17b)$$

□

B Additional experimental results

B.1 Results of the layer-wise parameterization

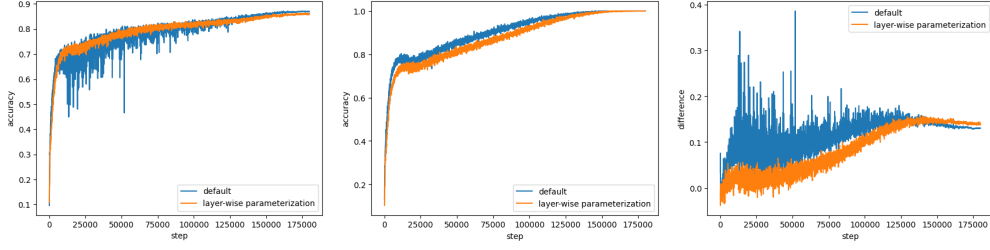


Figure B.1: The plots of the results of the training with layer-wise parameterization from Section 3.2.6 in comparison with the default setting, for 180,000 steps. To the left is the plot of train accuracy, to the center is the plot of the test accuracy, and to the right is the plot of the difference between the two accuracies.